

## Graphs and Trees Review

### Graphs Overview

A graph is a set of nodes (vertices) connected by edges (connections). Each node can connect to multiple other nodes, forming a network. The different types of graphs include:

- Directed: edges have a direction (from one node to another), often represented with straight lines and arrows.
- Undirected: edges have no direction, represented with simple lines.
- Vertex: each node is associated to a label or identifier.
- Cyclic: contains cycles (a path that starts and ends at the same node).
- Acyclic (DAG): does not contain cycles.
- Edge labeled: each edge has a label usually drawn next to corresponding edge.
- Weighted: edges have weights (values) associated with them, that can be used to perform arithmetic operations.
- Disconnected: contains two or more nodes that are not connected by any edges.

Graphs are used in various applications such as maps, networks, recommendation systems, dependency resolution.

### Graph Traversals

This involves visiting all the nodes in a graph. The two main algorithms are:

- **Breadth-First Search (BFS)**
  - Uses a queue.
  - Explores level by level.
  - Finds shortest path in unweighted graphs.
- **Depth-First Search (DFS)**
  - Uses a stack (or recursion).
  - Explores a branch fully before backtracking.
  - Useful for cycle detection and path finding.

### Graph Representations

Graphs can be represented in two main ways:

- **Adjacency List**
  - Each node has a list of its neighbors.
  - Space efficient for sparse graphs.
  - Easy to iterate over neighbors.
- **Adjacency Matrix**
  - A 2D array where rows and columns represent nodes.
  - Space intensive for large graphs.
  - Fast to check if an edge exists between two nodes.

### Trees

A tree is a special type of graph that is acyclic and connected. Key properties include:



- Binary Trees
  - Each node has at most two children, a left and a right child.
- Binary Search Trees (BST)
  - A binary tree in which every left child is less than its parent, and every right child is greater than its parent.

## Tries

Also known as prefix trees, they are used to store sets of strings, where each node represents a character.

Shared prefixes are stored only once, making them efficient for tasks like autocomplete and spell checking.

Search and insertion operations have a time complexity of  $O(L)$ , where  $L$  is the length of the string.

## Priority Queues

A priority queue is an abstract data type where each element has a priority.

Queues and stacks consider only the order of insertion, while priority queues consider the priority of elements.

Standard queues follow FIFO (First In First Out) and stacks follow LIFO (Last In First Out). However, in a priority queue, elements with higher priority are served before those with lower priority, regardless of their insertion order.

## Heaps

It's a specialized tree-based data structure with a very specific property called the heap property.

The heap property determines the relationship between parent and child nodes. There are two types of heaps:

- Max-Heap
  - The value of each parent node is greater than or equal to the values of its children.
  - The largest element is at the root.
- Min-Heap
  - The value of each parent node is less than or equal to the values of its children.
  - The smallest element is at the root.

### Python `heapq` module example

#### ▼ Example Code

```
import heapq

# Create empty heap
my_heap = []

# Insert elements
heapq.heappush(my_heap, 9)
heapq.heappush(my_heap, 3)
heapq.heappush(my_heap, 5)
print(my_heap) # [3, 9, 5]
```



```
# Push + Pop in one step
print(heapq.heappushpop(my_heap, 7)) # 5
print(my_heap) # [7, 9]

# Transform list into heap
nums = [5, 7, 3, 1]
heapq.heapify(nums)
```

## Using Priorities

### ▼ Example Code

```
my_heap = []
heapq.heappush(my_heap, (3, "A"))
heapq.heappush(my_heap, (2, "B"))
heapq.heappush(my_heap, (1, "C"))

# Removes lowest number = highest priority
print(heapq.heappop(my_heap)) # (1, "C")
```

### Assignment

☐ Review the Graphs and Trees topics and concepts.

Please complete the assignment

Submit

Ask for Help